

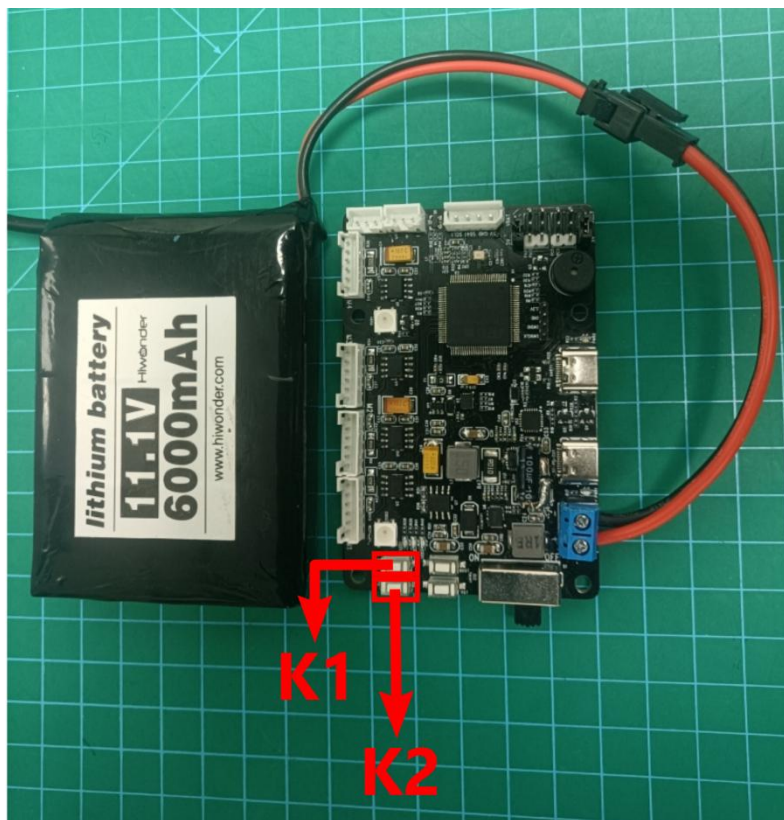
Lesson 3 RRCLite Button Detection Source Code Explanation

1. Program Logic

This chapter mainly focuses on controlling the buzzer sound through the built-in buttons on the STM32 main control board, accompanied by different LED status blinking.

2. Hardware Connection

The hardware connection diagram is as below:



3. Program Download

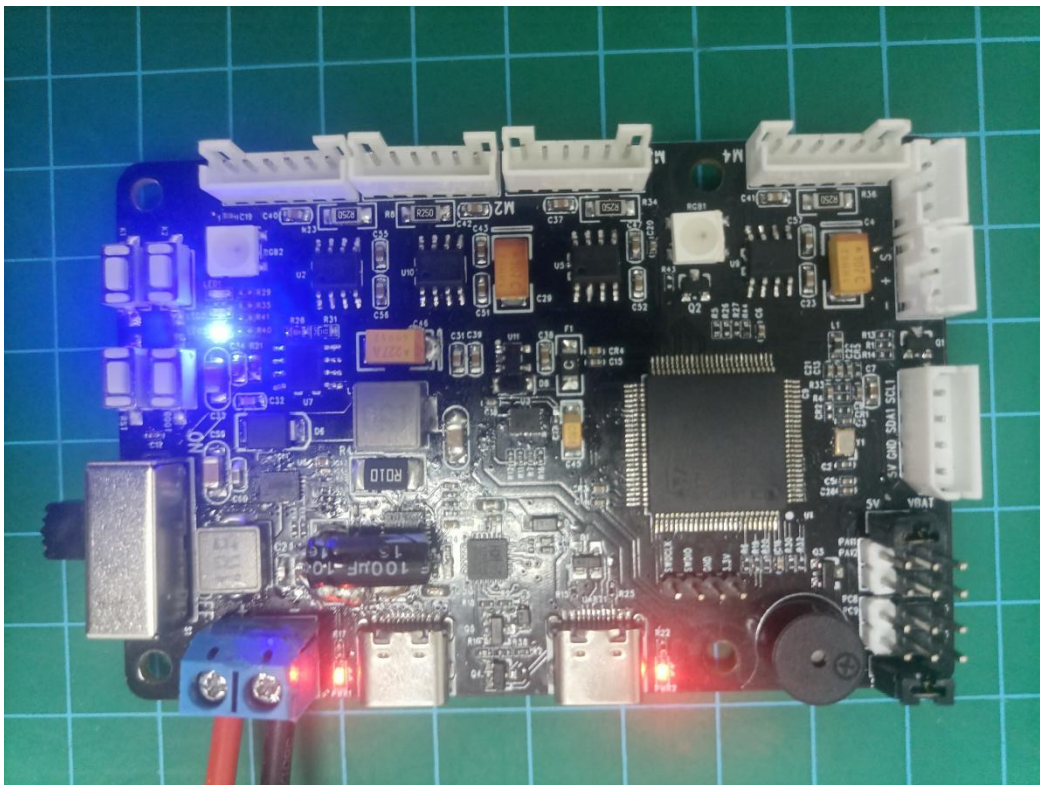
The example for this section is “rosrobotcontrollerm4_key_buzzer”.

For specific instructions on how to download the program, refer to the

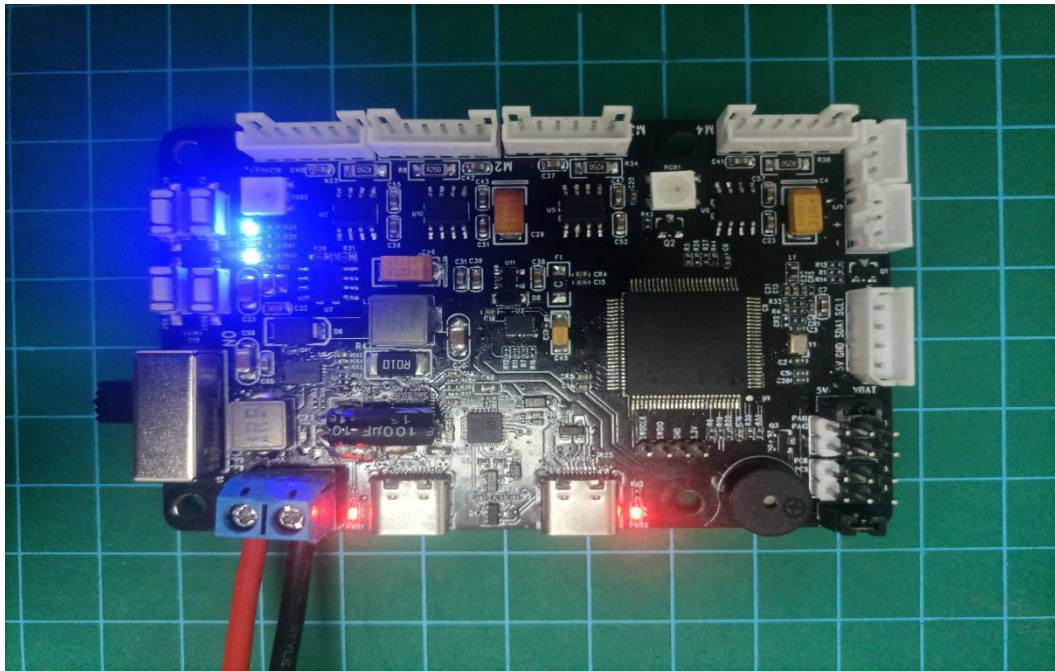
document: "2.STM32 Development Fundamentals/3.Project
Compilation and Download".

4. Program Outcome

1. Press the K2 button twice at startup, and the buzzer will emit three consecutive beeps.
2. Press the K1 button; upon a single press, the buzzer emits 3 beeps. After the first press, the LED turns off; after the second press, the LED turns on; after the third press, the LED starts blinking.



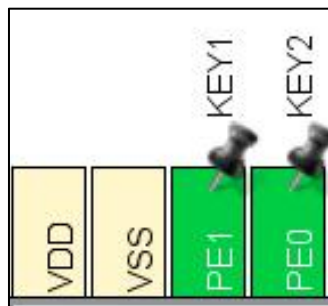
(The effect for the first time the K1 is clicked when power on)



(The effect for the second time the K1 is clicked when power on)

5. PIN Configuration

Using the STM32CubeMX software to check the pin information, according to the diagram below, it can be seen that the chip pin corresponding to the buzzer is PA8. The pin configuration corresponding to the buttons is PE1 and PE0.



6. Source Code Analysis

The source code for this lesson is located in the same folder as this tutorial

document, primarily analyzing the files Hiwonder/System/app.c and Hiwonder/Peripherals/buzzer.c.

6.1 LED State and Buzzer State Change

```
/* the frequency of buzzer */
#define BUZZER_FREQ 1300

void button_event_callback(ButtonObjectTypeDef *button, ButtonEventIDEnum event)
{
    if(button == buttons[0]) { /* the event of button1 */
        if(event == BUTTON_EVENT_CLICK) { //if it is a single click
            buzzer_didi(buzzers[0], BUZZER_FREQ, 100, 100, 3); //the buzzer sounds twice
            static uint8_t led_ctl = 0;
            switch(led_ctl)
            {
                case 0:
                    led_off(leds[0]); //LED off
                    led_ctl++;
                    break;
                case 1:
                    led_on(leds[0]); //LED on
                    led_ctl++;
                    break;
                case 2:
                    led_flash(leds[0],200,200,0); //LED flashing (on for 200ms, and off for 200ms)
                    led_ctl = 0;
                    break;
                default:
                    led_ctl = 0;
                    break;
            }
        }
    }
    if(button == buttons[1]) { /* the event of button2 */
        if(event == BUTTON_EVENT_DOUBLE_CLICK) { // if it is double click
            buzzer_didi(buzzers[0], BUZZER_FREQ, 100, 100, 3); // the buzzer sounds three times
        }
    }
}
```

Above is the App.c file, where button_event_callback() serves as the button callback function. The main programming flow is as follows:

Button event detection;

Different buzzer sounds for pressing button 1 and button 2;

Different LED state changes based on the number of times button 1 is pressed.

6.2 Buzzer Activation, Deactivation, and Setting of Beep

Duration

```
int buzzer_on( BuzzerObjectTypeDef *self, uint32_t freq)
{
    BuzzerCtrlTypeDef ctrl_structure = {
        .freq = freq,
        .ticks_on = 1,
        .ticks_off = 0, /* If the silent time is 0, the buzzer makes a long sound */
        .repeat = 0,
    };
    return self->put_ctrl_block(self, &ctrl_structure);
}

int buzzer_off(BuzzerObjectTypeDef *self)
{
    BuzzerCtrlTypeDef ctrl_structure = {
        .freq = 0,
        .ticks_on = 0, /* If the sound time is 0, it is silent */
        .ticks_off = 0,
        .repeat = 0,
    };
    return self->put_ctrl_block(self, &ctrl_structure);
}

int buzzer_didi(BuzzerObjectTypeDef *self, uint32_t freq, uint32_t ticks_on, uint32_t ticks_off, uint32_t repeat)
{
    BuzzerCtrlTypeDef ctrl_structure = {
        .freq = freq,
        .ticks_on = ticks_on,
        .ticks_off = ticks_off,
        .repeat = repeat,
    };
    return self->put_ctrl_block(self, &ctrl_structure);
}
```

Above is the buzzer.c file, where the buzzer_on() function is to turn on the buzzer. It initializes the buzzer control structure, sets the buzzer frequency, and sets other properties to make the buzzer continuously beep.

.ticks_off = 0, // Silence time is 0, indicating continuous buzzing

The buzzer_off() function is to turn off the buzzer.

.ticks_on = 0, // Beeping time is 0, indicating silence

The buzzer_didi() function is to set the buzzer's frequency and beep mode. It sets the buzzer's frequency, beeping time, silence time, and repeat count:

4freq, ticks_on, ticks_off, and repeat.

6.3 A Callback Function and Button Status Task are Registered

```
// register button callback function, and process button value
button_register_callback(buttons[0], button_event_callback);
button_register_callback(buttons[1], button_event_callback);
```

Above is the app.c file, where the button_register_callback function is used to register a callback function, button_event_callback, for specific buttons (here buttons[0] and buttons[1]). The callback function is invoked when specific events occur, such as pressing or releasing a button. When a button is pressed

or released, the callback function `button_event_callback` is executed, triggering the corresponding code to run.

```

84
85 void button_register_callback(ButtonObjectTypeDef *self, ButtonEventCallbackFuncTypeDef callback)
86 {
87     if(NULL == callback) {
88         return;
89     }
90     self->event_callback = callback;
91 }
92
93 void button_defalut_event_callback(ButtonObjectTypeDef *self, ButtonEventIDEnum event)
94 {
95 }
96
97 void button_object_init(ButtonObjectTypeDef *self)
98 {
99     self->stage = BUTTON_STAGE_NORMAL;
100     self->last_pin_raw = 0;
101     self->last_pin_filtered = 0;
102     self->combin_counter = 0;
103     self->ticks_count = 0;
104
105     /* config */
106     self->combin_th = 400;
107     self->lp_th = 2000;
108     self->repeat_th = 500;
109     self->event_callback = button_defalut_event_callback;
110 }

```

Above are the functions in the `button.c` file, including the callback function for registering button events and the default button event callback function.

```

void buzzer_task_handler(BuzzerObjectTypeDef *self, uint32_t period)
{
    /* Attempt to retrieve new control data from the queue. If it is successful, reset the state machine and start a new control loop */
    if(self->get_ctrl_block(self, &self->ctrl_structure) == 0) {
        self->stage = BUZZER_STAGE_START_NEW_CYCLE;
    }
    /* State machine processing */
    switch(self->stage) {
        case BUZZER_STAGE_START_NEW_CYCLE: {
            if(self->ctrl_structure.ticks_on > 0 && self->ctrl_structure.freq > 0) {
                self->set_pwm(self, self->ctrl_structure.freq); /* buzzer sounds */
                if(self->ctrl_structure.ticks_off > 0) /* If the silence time is not 0, the buzzer beeps; otherwise, it sounds continuously */
                    self->ticks_count = 0;
                self->stage = BUZZER_STAGE_WAITTING_OFF; /* wait for the sound duration to end */
            } else {
                self->stage = BUZZER_STAGE_IDLE; /* Sound continuously and change to the idle state */
            }
        } else { /* If the silent time is 0, the buzzer makes a long sound */
            self->set_pwm(self, 0);
            self->stage = BUZZER_STAGE_IDLE; /* be silent and transite to the idle state */
            break;
        }
        case BUZZER_STAGE_WAITTING_OFF: {
            self->ticks_count += period;
            if(self->ticks_count >= self->ctrl_structure.ticks_on) { /* End the sound duration */
                self->set_pwm(self, 0);
                self->stage = BUZZER_STAGE_WAITTING_PERIOD_END;
            }
            break;
        }
        case BUZZER_STAGE_WAITTING_PERIOD_END: { /* Wait for the period to end */
            self->ticks_count += period;
            if(self->ticks_count >= (self->ctrl_structure.ticks_off + self->ctrl_structure.ticks_on)) {
                self->ticks_count = (self->ctrl_structure.ticks_off + self->ctrl_structure.ticks_on);
                if(self->ctrl_structure.repeat == 1) { /* End the control task if the remaining repeat count is 1 */
                    self->set_pwm(self, 0);
                    self->stage = BUZZER_STAGE_IDLE;
                } else {
                    self->set_pwm(self, self->ctrl_structure.freq);
                    self->ctrl_structure.repeat = self->ctrl_structure.repeat == 0 ? 0 : self->ctrl_structure.repeat - 1;
                    self->stage = BUZZER_STAGE_WAITTING_OFF;
                }
            }
            break;
        }
        case BUZZER_STAGE_IDLE: {
            break;
        }
        default:
            break;
    }
}

```

Above are the functions in the `button.c` file (partial screenshot omitted). The `button_task_handler` function handles different button states and events, including detecting button presses, releases, and long presses, and performs

corresponding actions as needed.

```
void button_timer_callback(void *argument)
{
    button_task_handler(buttons[0], BUTTON_TASK_PERIOD);
    button_task_handler(buttons[1], BUTTON_TASK_PERIOD);
}
```

The button_porting function periodically scans and updates the real-time status of the buttons.